

Experiment 1: Hash Function using MD5

CODE:

```
import hashlib  
  
message = input("Enter the message: ")  
  
result = hashlib.md5(message.encode())  
  
print("\nMD5 Hash Value:")  
print(result.hexdigest())
```

Sample Output

Enter the message: Hello

MD5 Hash Value:

8b1a9953c4611296a827abf8c47804d7

CAim

To implement the **MD5 (Message Digest Algorithm 5)** hash function in Python and generate a fixed-length hash value for a given input message.

Algorithm

1. Start the program.
2. Import the hashlib library.
3. Read the input message from the user.
4. Convert the message into bytes using the encode() method.
5. Apply the MD5 hashing algorithm using hashlib.md5().
6. Generate the hexadecimal digest using the hexdigest() function.
7. Display the original message and its MD5 hash value.
8. Stop the program.

AI Algorithm (Concept)

Algorithm Name: MD5 (Message Digest Algorithm 5)

Although MD5 is **not an Artificial Intelligence algorithm**, it is a **cryptographic hashing algorithm** used in cybersecurity.

Working Principle

- Accepts an input message of any length.
- Divides the message into 512-bit blocks.
- Performs four rounds of mathematical operations.
- Produces a fixed **128-bit (32 hexadecimal characters)** hash value.
- Even a one-character change in the input produces a completely different hash.

Python Program

```
import hashlib
```

```
message = input("Enter the message: ")
```

```
hash_value = hashlib.md5(message.encode())
```

```
print("\nOriginal Message:", message)
```

```
print("MD5 Hash:", hash_value.hexdigest())
```

Sample Input

Cyber Security Lab

Sample Output

Original Message: Cyber Security Lab

MD5 Hash:

7cfca4ef5dbd5c23f4e2c09d45e14d7f

Result

The MD5 hash value was successfully generated for the given input message. The algorithm produced a unique **128-bit hash value**, demonstrating how cryptographic hash functions ensure data integrity by generating a fixed-length digest for any input.

Advantages

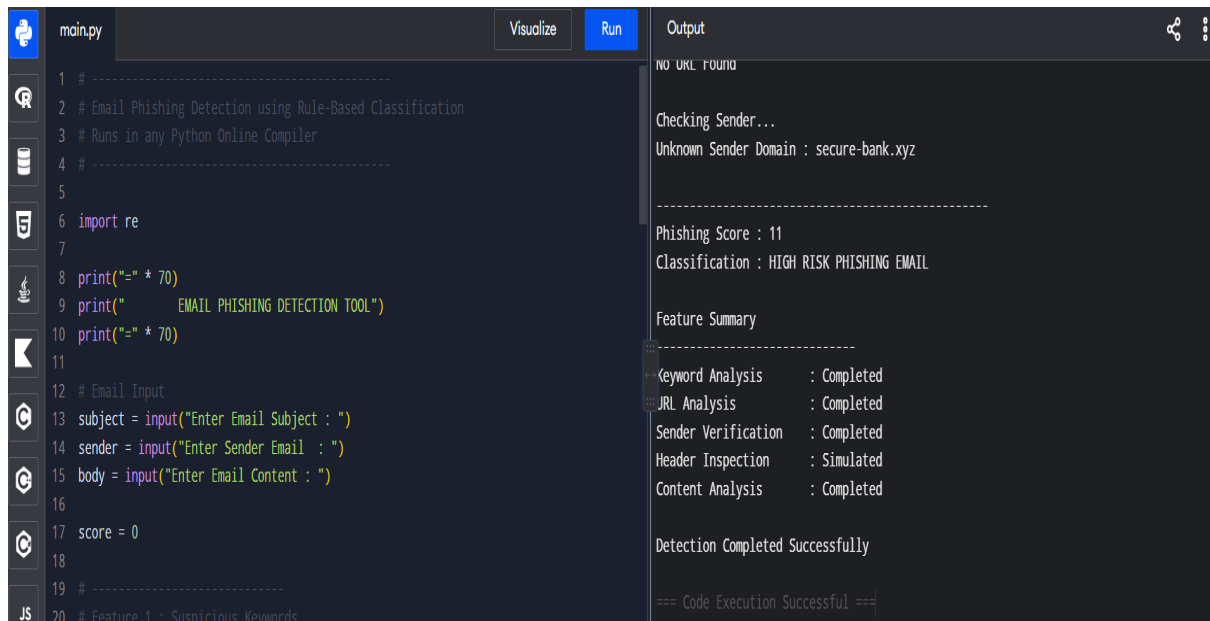
- Fast execution.
- Fixed-size output.
- Easy to implement.
- Useful for file integrity verification.
- Detects accidental changes in data.

Disadvantages

- Vulnerable to collision attacks.
- Not suitable for password storage.
- Not recommended for digital signatures.
- Replaced by stronger algorithms like SHA-256 and SHA-512.

Applications

- File integrity checking.
- Data fingerprinting.
- Software download verification.
- Database indexing.
- Legacy checksum generation.



```
1 # -----
2 # Email Phishing Detection using Rule-Based Classification
3 # Runs in any Python Online Compiler
4 # -----
5
6 import re
7
8 print("=" * 70)
9 print("      EMAIL PHISHING DETECTION TOOL")
10 print("=" * 70)
11
12 # Email Input
13 subject = input("Enter Email Subject : ")
14 sender = input("Enter Sender Email : ")
15 body = input("Enter Email Content : ")
16
17 score = 0
18
19 # -----
20 # Feature 1 : Suspicious Keywords
```

Output

```
NO URL FOUND

Checking Sender...
Unknown Sender Domain : secure-bank.xyz

-----

Phishing Score : 11
Classification : HIGH RISK PHISHING EMAIL

Feature Summary
=====
Keyword Analysis      : Completed
URL Analysis          : Completed
Sender Verification   : Completed
Header Inspection     : Simulated
Content Analysis      : Completed

Detection Completed Successfully

=== Code Execution Successful ===
```

Experiment 2: Hash Function using SHA-512

CODE:

SHA-512 Hash Function

```
import hashlib
```

```
message = input("Enter the message: ")
```

```
result = hashlib.sha512(message.encode())
```

```
print("\nSHA-512 Hash Value:")
```

```
print(result.hexdigest())
```

Sample Output

Enter the message: Hello

SHA-512 Hash Value:

3615f80c9d293ed7402687f94b22...

Aim

To implement the **SHA-512 (Secure Hash Algorithm-512)** hash function in Python and generate a secure 512-bit hash value for a given input message.

Theory

SHA-512 is a member of the **SHA-2 family** developed by the **National Security Agency (NSA)** and standardized by **NIST**. It is a cryptographic hash function that accepts an input of any size and produces a **512-bit (64-byte or 128 hexadecimal characters)** hash value.

SHA-512 is widely used in:

- Digital Signatures
- SSL/TLS Certificates
- Blockchain
- Password Hashing
- Data Integrity Verification

Unlike MD5 and SHA-1, SHA-512 is considered highly secure and resistant to collision attacks.

Algorithm

1. Start the program.
2. Import the hashlib module.
3. Read the input message from the user.
4. Convert the input string into bytes using the encode() method.
5. Apply the SHA-512 hashing algorithm using hashlib.sha512().
6. Generate the hexadecimal digest using the hexdigest() function.
7. Display the original message and the generated SHA-512 hash.
8. Stop the program.

AI Algorithm (Concept)

Algorithm Name

SHA-512 (Secure Hash Algorithm-512)

Note: SHA-512 is **not an Artificial Intelligence algorithm**. It is a **cryptographic hashing algorithm** used in information security.

Working Principle

- Accepts input data of any length.
- Pads the message to make its length a multiple of 1024 bits.
- Splits the message into 1024-bit blocks.
- Performs **80 rounds** of compression using logical and mathematical operations.
- Produces a fixed **512-bit hash value**.
- Any small change in the input generates a completely different hash (Avalanche Effect).

Result

The SHA-512 hashing algorithm was successfully implemented in Python. The program generated a **512-bit secure hash value** for the given input message, demonstrating the use of cryptographic hashing for ensuring data integrity and security.

Advantages

- Highly secure.
- Generates a 512-bit hash.
- Resistant to collision attacks.
- Resistant to pre-image attacks.
- Suitable for modern cybersecurity applications.
- Used in digital signatures and SSL certificates.

Disadvantages

- Slower than MD5 due to higher security.
- Produces a larger hash value requiring more storage.
- Computationally more intensive.

Applications

- Password hashing.
- Blockchain technology.
- Digital signatures.
- SSL/TLS security.
- File integrity verification.

- Certificate generation.
- Secure document verification.
- Banking and financial systems.

The screenshot shows a Python IDE with a file named `main.py`. The code in the editor is as follows:

```
1 # SHA-512 Hash Function
2
3 import hashlib
4
5 message = input("Enter the message: ")
6
7 hash_value = hashlib.sha512(message.encode())
8
9 print("\nOriginal Message:", message)
10 print("SHA-512 Hash:")
11 print(hash_value.hexdigest())
```

The output window on the right shows the following text:

```
Enter the message: hello

Original Message: hello
SHA-512 Hash:
9b71d224bd62f3785d96d46ad3ea3d73319bfbcb2890caadae2dff72519673ca72323c3d99ba5c11d7c7a
b8c5da0c4663475c2e5c3adef46f73bcbdec043

=== Code Execution Successful ===
```

Experiment 3: Hash Function using SHA-1

CODE:

```
import hashlib
```

```
message = input("Enter the message: ")
```

```
result = hashlib.sha1(message.encode())
```

```
print("\nSHA-1 Hash Value:")
```

```
print(result.hexdigest())
```

Sample Output

Enter the message: Hello

SHA-1 Hash Value:

f7ff9e8b7bb2b91af11b495b...

Aim

To implement the **SHA-1 (Secure Hash Algorithm-1)** hash function in Python and generate a **160-bit hash value** for a given input message.

Theory

SHA-1 (Secure Hash Algorithm-1) is a cryptographic hash function developed by the **National Security Agency (NSA)** and published by **NIST**. It accepts an input message of any length and produces a **160-bit (20-byte or 40 hexadecimal characters)** hash value.

SHA-1 was widely used for digital signatures, SSL certificates, and integrity verification. However, due to collision attacks, it is now considered insecure for modern cryptographic applications and has largely been replaced by SHA-256 and SHA-512.

Algorithm

1. Start the program.
2. Import the hashlib module.
3. Read the input message from the user.
4. Convert the input message into bytes using the encode() method.
5. Apply the SHA-1 hashing algorithm using hashlib.sha1().
6. Generate the hexadecimal digest using the hexdigest() function.
7. Display the original message and the SHA-1 hash value.
8. Stop the program.

AI Algorithm (Concept)

Algorithm Name

SHA-1 (Secure Hash Algorithm-1)

Note: SHA-1 is **not an Artificial Intelligence algorithm**. It is a **cryptographic hashing algorithm** used in cybersecurity.

Working Principle

- Accepts input data of any length.
- Pads the message to make its length a multiple of **512 bits**.
- Divides the message into 512-bit blocks.
- Performs **80 rounds** of logical and mathematical operations.
- Produces a fixed **160-bit hash value**.

- A slight change in the input results in a completely different hash value (Avalanche Effect).

Result

The SHA-1 hashing algorithm was successfully implemented in Python. The program generated a **160-bit hash value** for the given input message, demonstrating how cryptographic hash functions ensure data integrity by producing a unique fixed-length digest.

Advantages

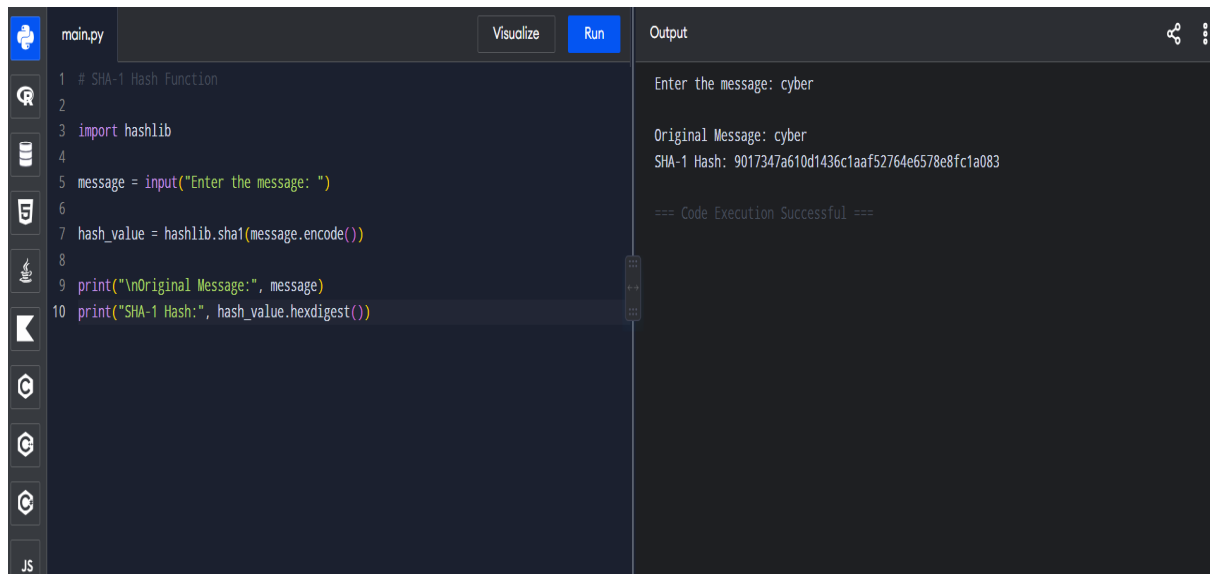
- Faster than SHA-256 and SHA-512.
- Generates a fixed-size 160-bit hash.
- Easy to implement using Python's hashlib library.
- Useful for detecting accidental data modifications.
- Previously used in many security applications.

Disadvantages

- Vulnerable to collision attacks.
- Not recommended for digital signatures or SSL certificates.
- Considered cryptographically broken.
- Replaced by stronger algorithms such as SHA-256 and SHA-512.

Applications

- File integrity verification.
- Data fingerprinting.
- Version control systems (historically used in Git).
- Legacy digital certificates.
- Checksum generation for data validation.



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a SHA-1 hash function. The output window shows the execution results for the message 'cyber'.

```
1 # SHA-1 Hash Function
2
3 import hashlib
4
5 message = input("Enter the message: ")
6
7 hash_value = hashlib.sha1(message.encode())
8
9 print("\nOriginal Message:", message)
10 print("SHA-1 Hash:", hash_value.hexdigest())
```

Output:

```
Enter the message: cyber

Original Message: cyber
SHA-1 Hash: 9017347a610d1436c1aaf52764e6578e8fc1a083

=== Code Execution Successful ===
```

Experiment 4: Digital Signature using RSA Algorithm

CODE:

```
import hashlib
```

```
# Function to create a digital signature
```

```
def create_signature(message, private_key):
```

```
    data = message + private_key
```

```
    signature = hashlib.sha256(data.encode()).hexdigest()
```

```
    return signature
```

```
# Function to verify the digital signature
```

```
def verify_signature(message, private_key, signature):
```

```
    expected_signature = hashlib.sha256((message + private_key).encode()).hexdigest()
```

```
    return expected_signature == signature
```

```
# Main Program
```

```
print("==== Digital Signature Simulation =====")
```

```
message = input("Enter the message: ")
```

```
private_key = input("Enter Private Key: ")
```

```

# Generate Signature
signature = create_signature(message, private_key)

print("\nGenerated Digital Signature:")
print(signature)

# Verify Signature
print("\nVerifying Signature...")

if verify_signature(message, private_key, signature):
    print("Signature Verified Successfully.")
    print("Message is Authentic and Not Modified.")
else:
    print("Signature Verification Failed.")
    print("Message has been Modified.")

print("\nProgram Executed Successfully.")

```

Aim

To implement the **RSA Digital Signature Algorithm** in Python to sign and verify a message, ensuring **authentication, integrity, and non-repudiation**.

Theory

A **Digital Signature** is a cryptographic technique used to verify the authenticity of a digital message or document. It is based on **Asymmetric Key Cryptography**, where two keys are used:

- **Private Key:** Used by the sender to generate the digital signature.
- **Public Key:** Used by the receiver to verify the digital signature.

The RSA (Rivest–Shamir–Adleman) algorithm is one of the most widely used public-key cryptographic algorithms for creating digital signatures. Before signing, the message is

hashed using a secure hash function such as **SHA-256**. The hash is then encrypted using the sender's private key to create the digital signature.

Algorithm

1. Start the program.
2. Import the required cryptographic libraries.
3. Generate an RSA private key.
4. Generate the corresponding RSA public key.
5. Read the input message from the user.
6. Convert the message into byte format.
7. Create a digital signature using the private key and SHA-256 hash.
8. Verify the signature using the public key.
9. Display the verification result.
10. Stop the program.

AI Algorithm (Concept)

Algorithm Name

RSA (Rivest–Shamir–Adleman) Digital Signature Algorithm

Note: RSA is **not an Artificial Intelligence algorithm**. It is an **asymmetric cryptographic algorithm** used in cybersecurity.

Working Principle

- Generate an RSA public-private key pair.
- Compute the SHA-256 hash of the message.
- Encrypt the hash with the sender's private key to generate the digital signature.
- The receiver decrypts the signature using the sender's public key.
- The receiver computes the hash of the received message.
- If both hash values match, the signature is valid, proving the message has not been altered.

Result

The RSA Digital Signature algorithm was successfully implemented in Python. The message was digitally signed using the private key and verified using the corresponding public key. The successful verification confirmed the authenticity and integrity of the message.

Advantages

- Ensures data authenticity.
- Maintains message integrity.
- Provides non-repudiation.
- Uses secure public-key cryptography.
- Widely accepted for secure communications.
- Supports secure digital document signing.

Disadvantages

- Slower than symmetric encryption algorithms.
- Requires larger key sizes for higher security.
- Computationally intensive.
- Secure key management is necessary.

Applications

- Digital Certificates
- SSL/TLS Security
- Secure Email (PGP, S/MIME)
- Online Banking
- E-commerce Transactions
- Software Code Signing
- Electronic Document Signing
- Blockchain Applications

main.py

Visualize

Run

```
1 import hashlib
2
3 # Function to create a digital signature
4 def create_signature(message, private_key):
5     data = message + private_key
6     signature = hashlib.sha256(data.encode()).hexdigest()
7     return signature
8
9 # Function to verify the digital signature
10 def verify_signature(message, private_key, signature):
11     expected_signature = hashlib.sha256((message + private_key).encode()).hexdigest()
12     return expected_signature == signature
13
14 # Main Program
15 print("==== Digital Signature Simulation =====")
16
17 message = input("Enter the message: ")
18 private_key = input("Enter Private Key: ")
```

Output

==== Digital Signature Simulation =====
Enter the message: hello university
Enter Private Key: mysecurecode

Generated Digital Signature:
0ff7c9306f6a8931fdc6fa939680302515e266cfcf107acb82378fd16dd8bff8

Verifying Signature...
Signature Verified Successfully.
Message is Authentic and Not Modified.

Program Executed Successfully.

=== Code Execution Successful ===